```
   def apply(): Behaviors.Receive[Command] =
      Behaviors.receive { (context, message) =>
        message match {
          case CreateChild(name) =>
            context.spawn(SimplifiedWorker(), name)    ❷
            Behaviors.same
}
```

❶ Protocol

❷ Spawns the actor

Now let's focus on testing:

1. Create the test kit with `BehaviorTestKit(initialBehavior: Behavior[T])` so that `initialBehavior` is the actor you can test.
2. Test the expected effect with `testkit.expectedEffect(effect: Effect)`.
3. To send messages to the actor (`initialBehavior`), you can use `testkit.run(message: T)`: in this case, `CreateChild`.

The following listing uses the `BehaviorTestKit` to test whether an actor has been spawned. It proves that you don't get any effect before you send the message to the actor and that you get the spawned effect afterward.

Listing 4.1 Simplified manager testing spawning

```
import akka.actor.testkit.typed.scaladsl.BehaviorTestKit
import akka.actor.testkit.typed.Effect.{ NoEffects, Scheduled, Spawned }
import org.scalatest.wordspec.AnyWordSpec
import org.scalatest.matchers.should.Matchers

class SyncTestingSpec extends AnyWordSpec with Matchers {

  "Typed actor synchronous testing" must {
```